



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

TORNADO PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on IOloop, async handlers, WebSockets, and testing

TOTAL: 10 QUESTIONS

Q1 What is Tornado and what makes it suited for high-concurrency use cases?

Threat Model

Q6 How does Tornado's template engine handle auto-escaping?

Security

Q2 How does the Tornado IOloop work with modern Python architecture and asyncio?

Architecture

Q7 How do you implement user authentication in Tornado?

Lifecycle

Q3 How do you define a request handler in Tornado?

Configuration

Q8 How do you enable SSL/TLS in Tornado's HTTPServer?

Compliance

Q4 What is the difference between `@tornado.gen.coroutine` and `async/await`?

Language

Q9 How do you run Tornado across multiple CPU cores?

Testing

Q5 How does Tornado's WebSocket handler work?

SSO / IdP

Q10 How do you write async tests in Tornado?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Tornado is a Python web framework and

Mitigation

Tornado is a Python web framework and async networking library with a non-blocking I/O model. A single-threaded IOLoop handles thousands of concurrent connections efficiently, avoiding the thread-per-request overhead of traditional WSGI servers.

A6 Tornado templates auto-escape HTML output by default

Security, Vulnerability

Tornado templates auto-escape HTML output by default, preventing XSS. Use `{% raw expr %}` to render unescaped content. Unlike Jinja2, Tornado templates are compiled to Python code, making them fast but with a smaller feature set.

A2 Since Tornado 5, the IOLoop is built

Primitives

Since Tornado 5, the IOLoop is built on top of `asyncio`'s event loop. `tornado.ioloop.IOLoop.current()` returns the current loop. You can mix Tornado coroutines with native `asyncio` code. `asyncio.run()` starts the loop for both frameworks simultaneously.

A7 Override `get_current_user()` to return the authenticated user

Authentication

Override `get_current_user()` to return the authenticated user (e.g., decoded from a signed cookie with `self.get_secure_cookie()`). Decorate handlers with `@tornado.web.authenticated`, which redirects unauthenticated users to `login_url`.

A3 Subclass `tornado.web.RequestHandler` and override HTTP

Hardening

Subclass `tornado.web.RequestHandler` and override HTTP method coroutines: `async def get(self)`, `async def post(self)`. Use `self.get_argument()`, `self.request.body`, and `self.write()` to read input and write output. Map it to a URL in the Application URL spec.

A8 Pass `ssl_options={'certfile': '/path/cert.pem', 'keyfile': '/path/key.pem'}` to

Governance

Pass `ssl_options={'certfile': '/path/cert.pem', 'keyfile': '/path/key.pem'}` to `HTTPServer(app, ssl_options=...)`. `HTTPServer` can also accept an `ssl.SSLContext` object for advanced configuration like client certificate verification.

A4 `@gen.coroutine with yield` was Tornado's pre-3.5 `async`

Gotchas

`@gen.coroutine with yield` was Tornado's pre-3.5 `async` style and still works. Native `async def / await` is preferred in Python 3.5+ and is more readable. Under the hood both produce the same IOLoop-driven coroutine, so they interoperate freely.

A9 Call `tornado.process.fork_processes(0)` to fork one worker per

Worker, per

Call `tornado.process.fork_processes(0)` to fork one worker per CPU core. Each process runs its own IOLoop. Alternatively, start multiple Tornado processes behind a load balancer. Tornado's non-blocking model means fewer processes are needed than with WSGI.

A5 Subclass `WebSocketHandler`. Override `open()` (connection

Connection

Subclass `WebSocketHandler`. Override `open()` (connection established), `on_message(message)` (data received), and `on_close()` (connection closed). Call `self.write_message(data)` to send to the client. Map the handler to a URL like any `RequestHandler`.

A10 Subclass `AsyncHTTPTestCase` and override `get_app()` to

Configuration

Subclass `AsyncHTTPTestCase` and override `get_app()` to return your Application. Use `self.fetch('/path')` which blocks until the response arrives. `@gen_test(timeout=5)` decorates test methods that are coroutines, running them on the IOLoop.